

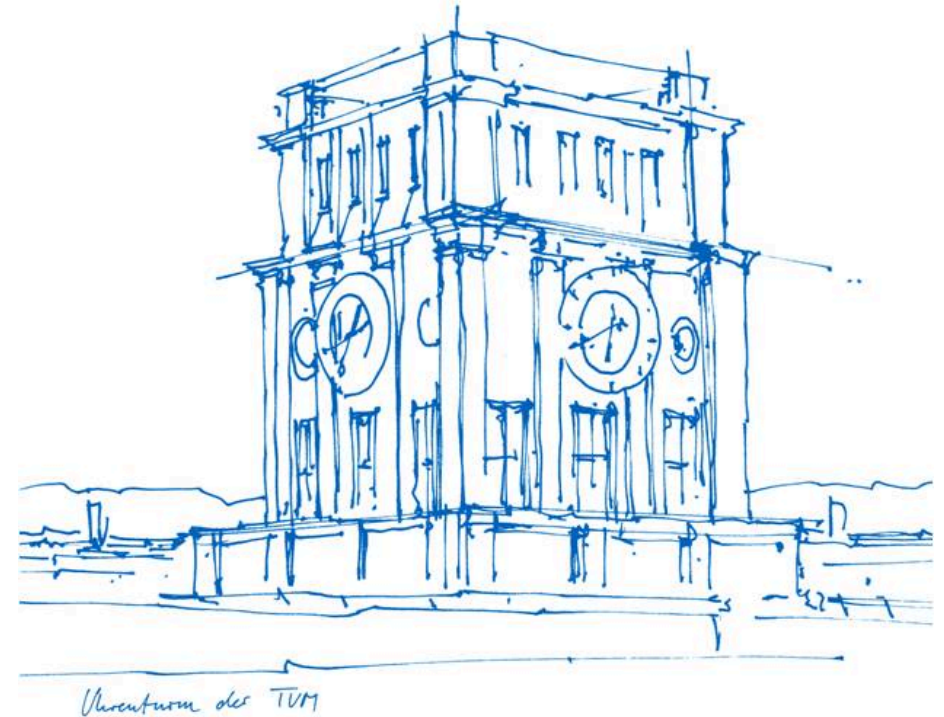
Agentic AI in Modern Business

A GUI agent for AndroidWorld, and where it breaks

Group project

Jonas Schmidt · Vincent Fischer · Liam Meinhart
Bela Gehlen · Gloria Dell'Etere

Technische Universität München · Campus Heilbronn
Agentic AI in Modern Business · Group Project · 09.10.2026



AGENDA

Five chapters, one story

- 1 The assignment**
What we were asked to build, and the three tasks we chose to run it on.
- 2 How we built it**
One agent, three versions. The decisions behind it, from the model to the guardrails.
- 3 How we measured**
When a run stops, what counts as a pass, and the one row we wrote per run.
- 4 What happened**
The V1 numbers, one failure step by step, and the diagnosis.
- 5 What we changed and learned**
One change, the before-and-after numbers, and where a fix belongs.

1

The assignment

What we were asked to build, and the three tasks we chose to run it on.

Build an agent that operates a phone, measure it, improve it

THE TASK

- A GUI agent for AndroidWorld
- One class, one `step()`
- The agent only sees pixels and buttons

THE GOAL

- Not a demo: a measured result
- 3 tasks × 3 runs × 3 versions
- Failures explained, every version measured before and after

THE GUIDELINES

- Any model, but name it
- Guardrails in code, not in the prompt
- One task per difficulty row

The three tasks: a contact, a note, a note sent by SMS

Difficulty	Task	What the agent is told	Budget
Warm-up	ContactsAddContact	Create a new contact for {name}. Their number is {number}.	12
Real work	MarkorCreateNote	Create a new note in Markor named {file_name} with the following text: {text}	16
Multi-app & memory	MarkorCreateNoteAndSms	Create a note named {file_name} with {text}. Share its entire content with {number} via SMS.	18

2

How we built it

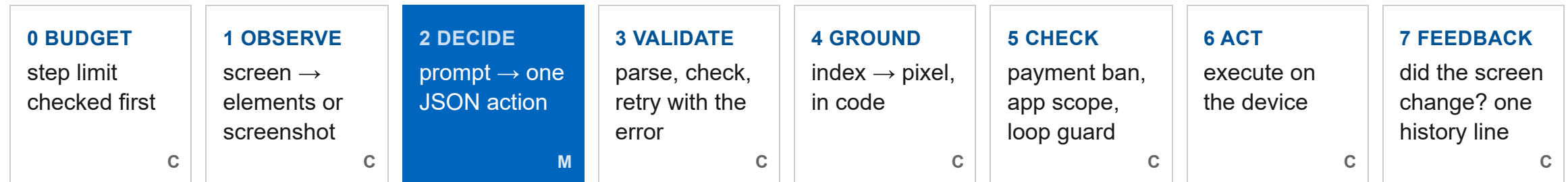
One agent, three versions. The decisions behind it, from the model to the guardrails.

Which model we use and how it is configured

Model	qwen3-vl:4b-instruct, 4.4B parameters
Where it runs	laptop, Ollama server at localhost:11434
Quantisation	Q4_K_M, 4 bits per weight
Memory for weights	$4.4\text{B} \times 0.5 \text{ byte} + \text{projector} \approx \mathbf{3.3 \text{ GB}}$
Context window	num_ctx 8,192 tokens (Ollama default 4,096 is too small for tree + image)
Sampling	temperature 0 , seed 42 , top-p left at default
Output	max 400 tokens, JSON-schema constrained

- Free after download, slow, small only
- Temperature 0 + fixed seed, so a result can be repeated
- The variant that answers at once, not the one that thinks first
- Every run records name, size, quantisation and digest

Agent design: the eight stages of one step()



C = code we wrote · M = the model

Harness part	What it decides	Where in step()
Loop control	when to call the model again, when to stop	stage 0, stop rules
Context assembly	what goes into the window, inside the budget	stage 1, prompts.py, memory.py
Tool dispatch	turn the JSON into a real tap on the device	stages 4 and 6
Error recovery	parse failed, no effect: retry, repair or give up	stages 3 and 7
Guardrails	what is forbidden, what gets logged	stage 5, guardrails.py

HOW WE BUILT IT · ONE STEP, TRACED

One step of the ReAct loop: reason → act → observe

```
REASON "thought": "I need to create a new contact for
        Hugo Pereira. The 'Create contact' button is
        visible and accessible."
ACT     "action_type": "click", "index": 1
OBSERVE screen changed → new tree, 12 elements
        history: 5. click [1] "Create contact" -> executed
```

Step 5 of our ContactsAddContact run. One model call, one action, one observation, then the next call with a longer history.

Phase	Term	In our agent
Reason	no tool	the model's thought, logged
Act	Execution tools	click, long_press, input_text, scroll, open_app, back, home, enter
Act	Collaboration, user communication	none; infeasible is the only way to give up
Observe	Perception tools	read the screen: tree or screenshot
Act + Observe	Plugin ("Computer Use")	AndroidWorld supplies both spaces
before the loop	Tool discovery, MCP	not needed: ten fixed actions

Pixels see everything but point badly. The tree is precise but incomplete.

Screenshot V1

- Sees every pixel, also WebViews and drawings
- ~1,900 prompt tokens per step
- The model has to find the row itself

Accessibility tree V2 V3

```
[1] ImageButton "Create contact"  
[2] Button "Don't allow"  
[7] EditText "First name"  
[10] EditText "Phone"
```

- ~980 prompt tokens per step, exact bounds
- Whatever has no description is simply not there

How the agent points: coordinates vs. index

By coordinate v1

```
{"action_type": "click", "x": 540, "y": 735}
```

The model produces the numbers. A 20-pixel error is a tap on the gap between two rows, and nothing reports an error.

By index v2 v3

```
{"action_type": "click", "index": 3}
```

Our code looks element 3 up in the tree. It either finds it or it doesn't: a failure we can see and log.

Old screens are never resent: one line per step, status last

```
system  role · action space · rules      (stable prefix, cached)
user    GOAL: Create a new contact for Hugo Pereira ...
        HISTORY:
        1. navigate_home -> executed
        2. click [6] "Search" -> BLOCKED: home screen
        3. open_app contacts -> executed
        CURRENT SCREEN: [0] ... [13]      (this step only)
        STATUS: step 4 of 12 · app: contacts
        note: number +13920741751        (most volatile, last)
```

- A tool-call loop, but compressed
each step gets a fresh prompt: the old screen is dropped, one text line per step stays. 20 kept screenshots would cost 20,000+ tokens
- Stable prefix first, volatile last, so the KV cache hits
the model reuses what it computed for an unchanged beginning: fixed rules first, the ever-changing status line last
- History and status are written by code, not by the model
"BLOCKED" or "changed nothing" is recorded by our code, so it is a fact. In V3 the note line carries a value into the next app

The action space: ten actions, one schema

click · long_press · input_text · scroll
open_app · navigate_back · navigate_home
keyboard_enter · wait · status

```
"action_type": {"enum": [the ten above]}  
"index":      {"type": "integer"}  
"app_name":   {"enum": ["contacts", "markor",  
                        "simple sms messenger", ...]}  
"goal_status": {"enum": ["complete", "infeasible"]}
```

- Schema handed to Ollama for constrained decoding
- open_app only by exact name from the allow-list
- Tolerant parsing, strict validation, one retry with the error

Guardrails: in the code, before the action, no off switch

- Step limit
- Payment ban, layer 1: app scope
- Payment ban, layer 2: deny rules
- Loop guard **V3**

A guardrail firing in our run **V2**

2. click [6] "Search"
-> **BLOCKED: do not tap or type on the home screen.**
Open the app you need with open_app and its exact name.
3. open_app contacts -> executed

Step 2: the model wanted to tap the home-screen search. The code refused, and told the model why. Step 3 did it right.

3

How we measured

When a run stops, what counts as a pass, and the one row we wrote per run.

HOW WE MEASURED · WHEN A RUN STOPS, AND WHAT COUNTS

A run ends for one of five reasons. Only one of them can be a PASS.

Stop reason	Who decided	Can be PASS?
model_done	model says status: complete	yes, if the verifier agrees
model_infeasible	model gives up	no
step_limit	code: budget reached	no, even if the screen is right
loop_abort	code: same action, same screen, 3× blocked (V3)	no
error	emulator or model server broke	ERROR, excluded from rates

PASS means both:

- the agent itself reported *complete*
- AndroidWorld's checker returns 1.0 on the device state

Running out of steps is never counted as done, and a correct screen without the report is still a FAIL: the agent has to know it is done.

HOW WE MEASURED · ONE ROW PER RUN

Written while the run happens, in the lecture's format

task	ContactsAddContact
run	1 of 3
model	qwen3-v1:4b-instruct (Q4_K_M, local, Ollama)
observation	accessibility tree
grounding	by index
steps	11 (budget 12)
verifier	FAIL
failure_class	lost_value
recording	runs/v2_index/ContactsAddContact/run1/recording.mp4
note	typed "Hugo", never "Pereira"; reported complete anyway

Written into `runs/log.csv` the moment a run ends, with a suggested failure class. A human confirms it from the recording (`failure_reviewed = yes`).

Failure class: one of five words, fixed list, one per run
grounding · too_early · lost_value · wrong_app · false_done

4

What happened

The V1 numbers, one failure step by step, and the diagnosis.

WHAT HAPPENED · RESULTS, V1 BASELINE

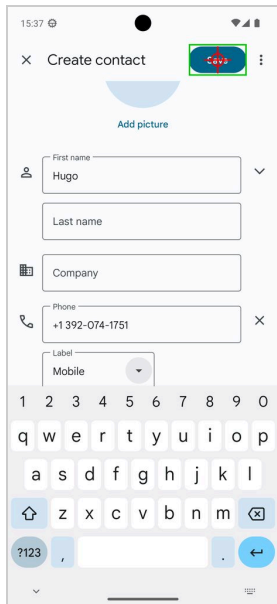
V1 results: 5 of 9 pass, every failure is in the two harder tasks V1

Task	Run	Steps (budget)	Verifier	Failure class	Note
ContactsAddContact	1/3	9 (12)	PASS		
ContactsAddContact	2/3	8 (12)	PASS		
ContactsAddContact	3/3	9 (12)	PASS		
MarkorCreateNote	1/3	10 (16)	PASS		
MarkorCreateNote	2/3	11 (16)	PASS		
MarkorCreateNote	3/3	9 (16)	FAIL	false_done (?)	reported complete without ever tapping Save
MarkorCreateNoteAndSms	1/3	17 (18)	FAIL	grounding (?)	note saved (credit 0.5); 1 tap on no element in the SMS app
MarkorCreateNoteAndSms	2/3	18 (18)	FAIL	grounding (?)	note saved (credit 0.5); 5 taps on the same spot in the SMS app, no effect
MarkorCreateNoteAndSms	3/3	18 (18)	FAIL	grounding (?)	number never typed; 4 taps on the same spot; out of steps

Windows laptop, 08.10.2026, 0 ERROR runs. (?) = suggested by the runner, to be confirmed against the recording.

WHAT HAPPENED · ONE FAILURE, STEP BY STEP

It typed "Hugo", never "Pereira", and said it was done V2



the 11 steps, one frame each

2 click [6] "Search" on the home screen → **blocked** by the harness

3 open_app contacts → executed

7 input_text "Hugo" into First name

9 input_text "+13920741751" into Phone

10 click [2] "Save"

11 status complete → verifier: **FAIL**

Model's own thought at step 11: "The contact has been successfully created."

Class: **lost_value**: a value from the goal never reached the device; the silent "done" is the symptom. Not a one-off: the surname is missing in 5 of 6 tree-based Contacts runs (V2, V3). V1 typed both names every time.

WHAT HAPPENED · WAS IT RELIABLE?

Reliability check: strong on constraints, weak on verification. V3 added it, and it misfired.

Reliability lever	What it asks	V1 / V2	V3
Goal clarity	What counts as success?	✓ the task text, plus AndroidWorld's checker on the device state	✓ same
Observable state	What can the agent see?	✓ the current screen every step; history lines show what changed	✓ same
Feedback	Does it know what happened?	weak : "executed"	"your action changed nothing": wrong on Save , Markor autosaves without a visible change
Verification	Did the result work?	none before done : the missing surname goes through	check step with the same 4B model: accepted the missing surname 3 of 3 times
Recovery	Retry, revise, roll back or stop?	invalid JSON: retry with the error, 2×. Timeout 300 s: ERROR	+ loop guard: same action, same screen, 3× blocked. Fired on Save , ended 3 runs
Constraints	Which rules must the system enforce?	✓ in code, not in the prompt: step limit, payment ban, app scope	✓ same, held in all 27 runs

Every V3 mechanism assumes that a successful action changes the screen. Where that assumption fails, the safety net turns a pass into a fail.

5

What we changed and learned

One change, the before-and-after numbers, and where a fix belongs.

WHAT WE CHANGED AND LEARNED · V1 → V2 → V3

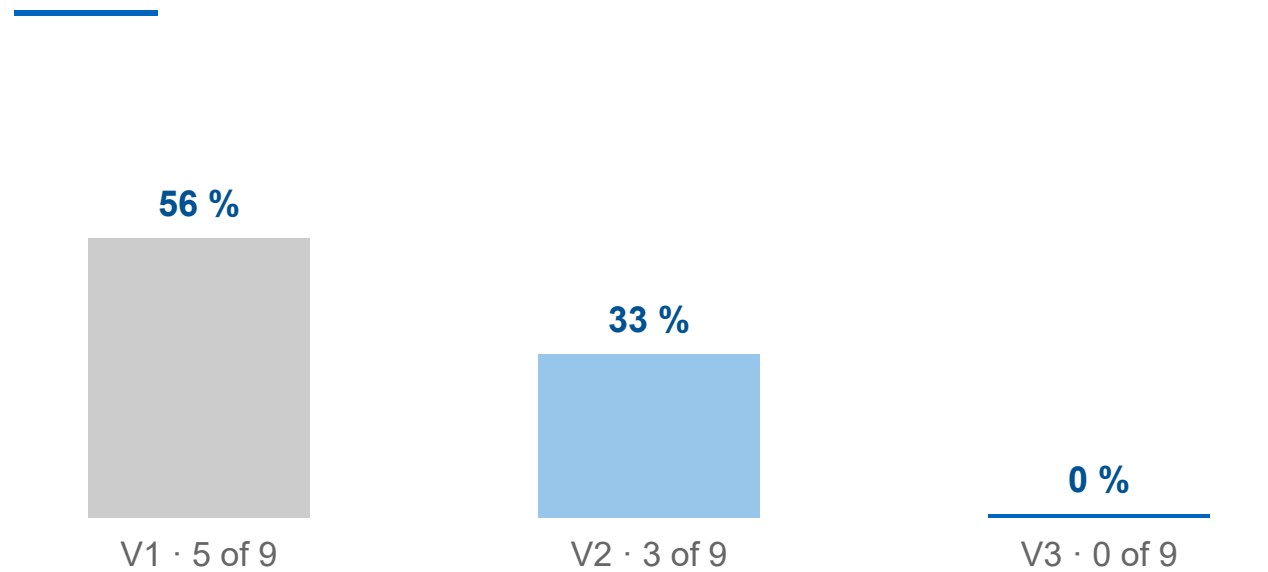
What changed between the versions: one change for V2, five more for V3

	V1 baseline V1	V2 index V2	V3 full harness V3
Observation	screenshot	accessibility tree	tree
Grounding	coordinates from the model	index, resolved in code	index
Harness additions	none	none	stable-screen wait · "no effect" hint · loop guard · status bar with notes · check before done
Model, temperature, seed	qwen3-vl:4b-instruct , Q4_K_M, local, T = 0, fixed seed	identical	identical
Tasks, task seeds, prompt structure	3 tasks, same 9 task instances	identical	identical
Guardrails	step limit, payment ban, app scope	identical	identical

V1 → V2 is one change, so its effect can be attributed. V2 → V3 stacks five, so it cannot: V3 is a package test.

WHAT WE CHANGED AND LEARNED · BEFORE / AFTER

V2 is 3× faster and cheaper, and less successful. V3 is worse again.



Success rate (verifier PASS), same 9 task instances per version, 0 ERROR runs.

	V1	V2	V3
model time per step	90 s	28 s	27 s
prompt tokens per call	1,905	1,069	1,120
time per run	19 min	7 min	8 min
invalid model replies	0	0	0

WHAT WE CHANGED AND LEARNED · WHERE THE FIX BELONGS

Where the fix belongs: code fixed pointing and app choice, not judgement

Failure	What we tried	Placed in	Did it work?	Where it really belongs
grounding	index instead of coordinates V2	Programs	yes : no more taps into nothing	Programs, as done
wrong_app	open_app by exact name only	Programs	yes : held in all 27 runs	Programs, as done
lost_value	status bar with notes V3	Programs	no : the surname was never read, not forgotten	Programs, but earlier: give the model tree + screenshot, or check the device state before done
false_done	check step before done V3	Programs	no : the checker was the same 4B model	Parameters: a stronger model for the check. Or Programs that read state the UI does not show
Save loop	"no effect" hint + loop guard V3	Programs	backfired : Markor autosaves, no visible change	Instructions: one sentence in a Markor skill file, "Save shows no confirmation"

The lecture's four places for a fix: Knowledge · Instructions · Programs · Parameters. "A rule that must never be bypassed → Programs." A judgement you can state in one sentence → Instructions.

WHAT WE CHANGED AND LEARNED · SAME CODE, SECOND MACHINE

Run again on a MacBook: same ranking, same failure patterns, different single runs

	Windows laptop	MacBook Air M4
V1 V1	5 / 9	5 / 9
V2 V2	3 / 9	2 / 9
V3 V3	0 / 9	0 / 9
ERROR runs	0	11, re-run
V1 model time per step	90 s	23 s
Ollama / model digest	0.40.0 / ef33995b	0.35.1 / ee4b975b

- Same script, same seeds, temperature 0
- Same patterns: the surname goes missing with the tree, V3 loops on Save
- Single runs differ: the model file is not the same build

"If you change quantisation, or switch model, re-run your evaluation. It is a different model." A different digest is a different model.

WHAT WE CHANGED AND LEARNED · [RUN IT YOURSELF](#)

One command per version. The config is the whole agent.

```
python run_eval.py --config configs/v2_index.json
```

- Same task seeds for every version (AndroidWorld's own seed formula)
- Temperature 0, fixed model seed; model name, size, quantisation and digest recorded per run
"if you change quantisation, or switch model, re-run your evaluation. It is a different model."
- AndroidWorld pinned to one commit; setup scripts for macOS and Windows; 209 unit tests without an emulator
- Every run leaves trajectory.json1, screenshots with the tap marked, recording.mp4, one log row

Four things the runs told us that a demo would not have

- **Where grounding happens matters more than prompt wording**

moving it into code made every step 3× faster and removed the taps into nothing. It also exposed the next weakness: lost values

- **Every safety mechanism encodes an assumption about the app**

V3's hint, loop guard and check step all assume a visible change. Markor autosaves. 2 of 9 became 0 of 9

- **A verifier as weak as the actor adds cost, not safety**

the check step was the same 4B model; it accepted the missing surname every time

- **Three runs per task find patterns, not rankings**

one run moves a task by 33 points; the trajectories carry the findings, not the rates

TAKEAWAY

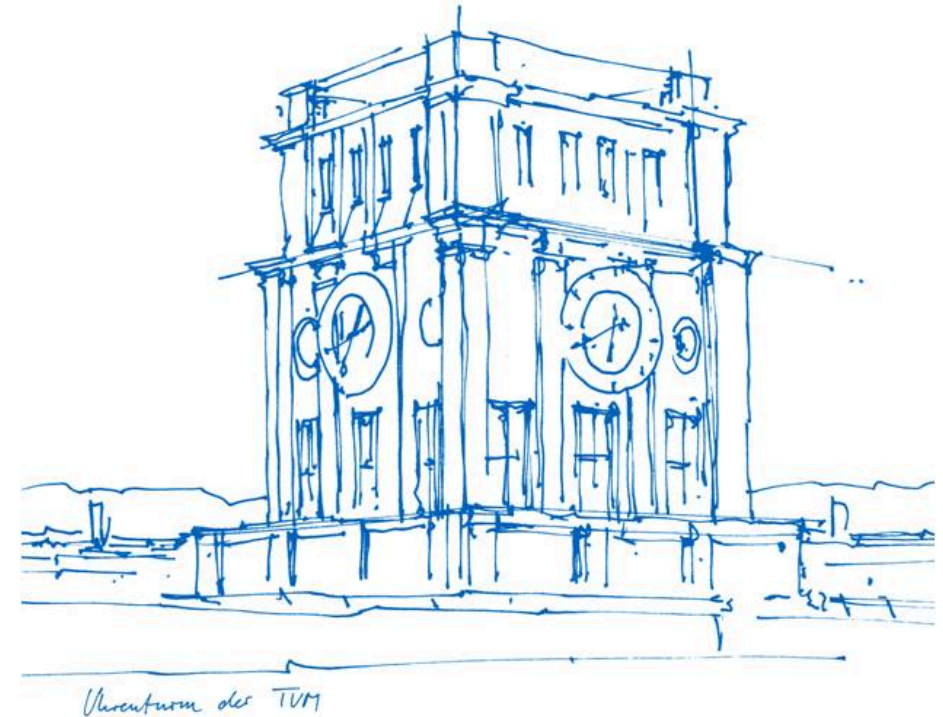
**The model decides. The
harness makes it safe,
measurable and honest.**

Thank you

Questions?

Jonas Schmidt · Vincent Fischer · Liam Meinhart
Bela Gehlen · Gloria Dell'Etere

Technische Universität München · Campus Heilbronn
Agentic AI in Modern Business · Group Project · 09.10.2026



Backup

V2 log rows · V3 · Skills, user memory, RAG · Reward design · Why we did not train

BACKUP · V2 INDEX (TREE + INDEX), 07.10.

V2 log rows

Task	Run	Steps (budget)	Verifier	Failure class	Note
ContactsAddContact	1/3	11 (12)	PASS		
ContactsAddContact	2/3	12 (12)	FAIL	grounding (?)	home-screen tap blocked; step limit
ContactsAddContact	3/3	9 (12)	FAIL	false_done (?)	reported complete, verifier says fail
MarkorCreateNote	1/3	16 (16)	FAIL	grounding (?)	state correct, never reported done
MarkorCreateNote	2/3	9 (16)	FAIL	grounding (?)	3 actions without effect; reported complete
MarkorCreateNote	3/3	9 (16)	PASS		
MarkorCreateNoteAndSms	1/3	16 (18)	FAIL	grounding (?)	partial credit 0.5; reported complete
MarkorCreateNoteAndSms	2/3	7 (18)	ERROR		emulator lost (adb: device not found), re-run
MarkorCreateNoteAndSms	3/3	0 (20)	ERROR		emulator lost, re-run

V3 stacks the remaining fixes. Several changes at once, so not attributable.

- Wait until the screen is stable before observing (`too_early`)
- "Your action changed nothing" feedback and a loop guard: same action on an unchanged screen is blocked the third time
- Status bar with scratch notes written by code (`lost_value`)
- Check step before done: the model is asked a second question, with a veto budget (`false_done`)

Results: [to be run]

Skills, user memory, RAG: what we built and what we left out

Concept	In our agent	Why
Skills	Built, switched off in all three versions. One tip file per app, added to the prompt only while that app is open	Tips cost tokens only when relevant. A tip changes the agent, so it would break the V1 → V2 comparison
User memory	Not built. The only memory is the note line inside one run V3	Every run starts fresh with new task values. Nothing is worth keeping between runs
RAG	Not built	Nothing to retrieve: the agent needs the current screen, and the tree delivers it every step

BACKUP · OUR VERIFIER IS A REWARD FUNCTION

The reward comes from the environment, never from the agent's own report

Reward question	Our answer
Source: who decides?	Verifiable rules: AndroidWorld's checker reads the device state. "The most reliable and the cheapest to run"
Information: how much?	One number, 1 or 0; the checker also gives partial credit (0.5 when the note exists but the SMS was not sent)
Timing: when?	Once, at the end of the run
Path: is every route acceptable?	An outcome reward is blind to the path, so we log what it cannot see: steps used, failure class, recording

46%

of a V2 run is waiting for the emulator, not the model (102 s per run, 55 s of it model time)

Reward hacking, our version: status: complete without checking is the boat going in circles. The agent tries it (false_done); the verifier does not pay for it.

BACKUP · WHY WE DID NOT TRAIN

Same loop as one RL training step. Our update went into the harness, not the weights.

One RL training step	Our evaluation
1 Attempts: 16 sandboxes	3 runs per task, fresh seeds
2 Reward: a script checks the final state	AndroidWorld's verifier, 1 or 0
3 Compare within the group	V1 vs. V2 on the same task instances
4 Update the weights	update the harness: one config change, re-run

What we observe	Rule of thumb	Us
Format or schema unstable?	SFT	no: 0 invalid replies, the schema does it
Domain knowledge missing?	Mid-training / retrieval	no: the screen is the knowledge
Non-zero success, reliable reward, good strategy rare	RL	yes, this is our case. But one attempt costs 2 to 7 minutes on a real emulator

"A cheap simulator favours classic RL; costly or irreversible interaction favours an LLM agent." And: "if your knowledge and your instructions are good enough, you do not need to do RL at all."